

I. Setting up the Python environment and libraries

AIM:

- Create a new notebook for Python
- Write and execute Python code
- Create new cells for code and Markdown
- Demonstrate the application of Jupyter Widgets

PROGRAM:

```
import ipywidgets as widgets

from IPython.display import display

slider = widgets.IntSlider(value=50, min=0, max=100, step=1,
description='Value:')

output = widgets.Output()

def on_value_change(change):
    output.clear_output()
    with output:
        print(f'Slider value is {change["new"]}')

slider.observe(on_value_change, names='value')

display(slider, output)
```

OUTPUT:



RESULT:

Thus, the given program was executed successfully.

II. EDA-Data Import and Export

AIM:

- Importing data from CSV, Excel, SQL databases, and web scraping
- Handling different data formats
- Export a DataFrame to an Excel file.

PROGRAM AND OUTPUT:

```
import pandas as pd
```

```
df_csv = pd.read_csv("/content/test_Y3wMUE5_7gLdaTN.csv")
```

```
df_csv.head()
```

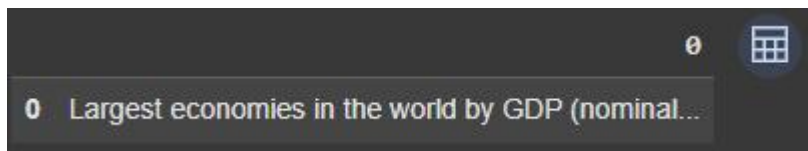
	Loan_ID	Gender	Married	Dependents	Education	Self_Employed	ApplicantIncome	CoapplicantIncome	LoanAmount	Loan_Amount_Term	Credit_History	Property_Area
0	LP001015	Male	Yes	0	Graduate	No	5720	0	110.0	360.0	1.0	Urban
1	LP001022	Male	Yes	1	Graduate	No	3076	1500	126.0	360.0	1.0	Urban
2	LP001031	Male	Yes	2	Graduate	No	5000	1800	208.0	360.0	1.0	Urban
3	LP001035	Male	Yes	2	Graduate	No	2340	2546	100.0	360.0	NaN	Urban
4	LP001051	Male	No	0	Not Graduate	No	3276	0	78.0	360.0	1.0	Urban

```
df_excel = pd.read_excel("/content/Historicalinvesttemp.xlsx",  
sheet_name="Sheet1")
```

```
df_excel.head()
```

	Unnamed: 0	Unnamed: 1	Unnamed: 2	Unnamed: 3
0	NaN	NaN	NaN	NaN
1	NaN	NaN	NaN	NaN
2	NaN	NaN	NaN	NaN
3	NaN	NaN	NaN	NaN
4	NaN	Annual Returns on Investments in	NaN	NaN
5	Year	Stocks	T.Bills	T.Bonds
6	1928	0.4381	0.0308	0.0084
7	1929	-0.083	0.0316	0.042
8	1930	-0.2512	0.0455	0.0454
9	1931	-0.4384	0.0231	-0.0256

```
url = "https://en.wikipedia.org/wiki/List_of_countries_by_GDP_(nominal)"
tables = pd.read_html(url)
df_web = tables[0]
df_web.head()
```



0 Largest economies in the world by GDP (nominal...

```
df=pd.read_excel("/content/Historicalinvesttemp.xlsx")
df.to_excel("/content/weeeeee.xlsx", index=False)
df.head()
```



	Unnamed: 0	Unnamed: 1	Unnamed: 2	Unnamed: 3
0	NaN	NaN	NaN	NaN
1	NaN	NaN	NaN	NaN
2	NaN	NaN	NaN	NaN
3	NaN	NaN	NaN	NaN
4	NaN	Annual Returns on Investments in	NaN	NaN

RESULT:

Thus, the given programs are written and executed successfully.

III.EDA-Data Cleaning

AIM:

- Handling missing values: detection, filling, and dropping
- Removing duplicates and unnecessary data
- Data type conversion and ensuring consistency
- Normalize data (e.g., standardization, min-max scaling).

PROGRAM AND OUTPUT:

```
# Importing required libraries
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.preprocessing import MinMaxScaler, StandardScaler

# Load the dataset
df = pd.read_csv('/content/test_Y3wMUE5_7gLdaTN.csv')

# Display basic information
print("Initial Data Overview:")
print(df.info())
```

```
Initial Data Overview:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 367 entries, 0 to 366
Data columns (total 12 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Loan_ID                367 non-null   object
1   Gender                 356 non-null   object
2   Married                367 non-null   object
3   Dependents             357 non-null   object
4   Education              367 non-null   object
5   Self_Employed          344 non-null   object
6   ApplicantIncome        367 non-null   int64
7   CoapplicantIncome      367 non-null   int64
8   LoanAmount             362 non-null   float64
9   Loan_Amount_Term       361 non-null   float64
10  Credit_History          338 non-null   float64
11  Property_Area          367 non-null   object
dtypes: float64(3), int64(2), object(7)
memory usage: 34.5+ KB
None
```

1. Handling Missing Values

```
print("\nMissing Values in Each Column:\n", df.isnull().sum())

sns.heatmap(df.isnull(), cbar=False, cmap="Blues")

plt.title("Missing Value Heatmap")

plt.show()

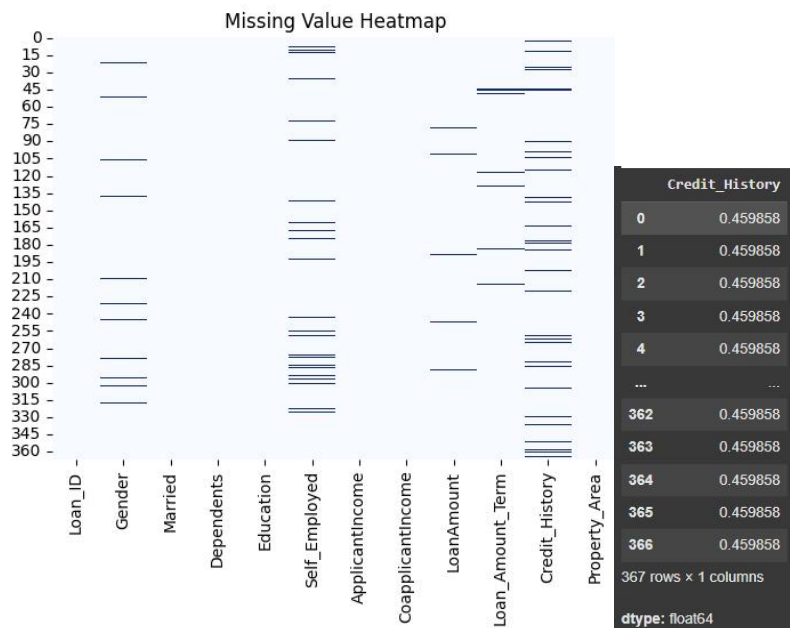
for col in ['Gender', 'Married', 'Dependents', 'Self_Employed']:
    df[col].fillna(df[col].mode()[0])

df['LoanAmount'].fillna(df['LoanAmount'].median())

df['Loan_Amount_Term'].fillna(df['Loan_Amount_Term'].mode()[0])

df['Credit_History'].fillna(df['Credit_History'].mode()[0])
```

```
Missing Values in Each Column:
Loan_ID                0
Gender                 11
Married                0
Dependents             0
Education              0
Self_Employed          23
ApplicantIncome        0
CoapplicantIncome      0
LoanAmount             5
Loan_Amount_Term       6
Credit_History         29
Property_Area          0
dtype: int64
```



2. Removing Duplicates

```
initial_rows = df.shape[0]
df.drop_duplicates(inplace=True)
print(f"\nRemoved {initial_rows - df.shape[0]} duplicate rows.")
```

```
Removed 0 duplicate rows.
```

3. Data Type Conversion

Convert 'Dependents' to numeric (replace '3+' with 3)

```
df['Dependents'] = df['Dependents'].replace('3+', 3).fillna(0).astype(int)
```

4. Ensuring Categorical Consistency

```
for col in ['Gender', 'Married', 'Education', 'Self_Employed', 'Property_Area']:
```

```
    df[col] = df[col].str.strip().str.capitalize()
```

5. Normalization

```
min_max_scaler = MinMaxScaler()
```

```
scale_cols = ['ApplicantIncome', 'CoapplicantIncome', 'LoanAmount']
```

```

df[scale_cols] = min_max_scaler.fit_transform(df[scale_cols])

scaler = StandardScaler()

df[['Credit_History']] = scaler.fit_transform(df[['Credit_History']])

# 6. Final Overview

print("\nCleaned Data Summary:")

print(df.info())

print(df.head())

```

```

Cleaned Data Summary:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 367 entries, 0 to 366
Data columns (total 12 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Loan_ID                367 non-null   object
1   Gender                 356 non-null   object
2   Married                367 non-null   object
3   Dependents             367 non-null   int64
4   Education              367 non-null   object
5   Self_Employed          344 non-null   object
6   ApplicantIncome        367 non-null   float64
7   CoapplicantIncome      367 non-null   float64
8   LoanAmount             362 non-null   float64
9   Loan_Amount_Term       361 non-null   float64
10  Credit_History         338 non-null   float64
11  Property_Area          367 non-null   object
dtypes: float64(5), int64(1), object(6)
memory usage: 34.5+ KB
None
   Loan_ID  Gender  Married  Dependents  Education  Self_Employed  \
0  LP001015  Male    Yes        0      Graduate         No
1  LP001022  Male    Yes        1      Graduate         No
2  LP001031  Male    Yes        2      Graduate         No
3  LP001035  Male    Yes        2      Graduate         No
4  LP001051  Male    No         0  Not graduate         No

   ApplicantIncome  CoapplicantIncome  LoanAmount  Loan_Amount_Term  \
0      0.078865      0.000000      0.157088      360.0
1      0.042411      0.062500      0.187739      360.0
2      0.068938      0.075000      0.344828      360.0
3      0.032263      0.106083      0.137931      360.0
4      0.045168      0.000000      0.095785      360.0

   Credit_History  Property_Area
0      0.459858      Urban
1      0.459858      Urban
2      0.459858      Urban
3      NaN      Urban
4      0.459858      Urban

```

RESULT:

Thus, the program was written and executed successfully.

IV. EDA-Data Inspection and Analysis

AIM:

- Viewing and inspecting DataFrames
- Filtering and subsetting data using conditions
- Descriptive statistics: measures of central tendency (mean, median, mode) and measures of dispersion (range, variance, standard deviation)

PROGRAM:

```
import pandas as pd

df = pd.read_csv('/content/test_Y3wMUE5_7gLdaTN.csv')

print(df.head())

print(df.tail())

print(df.info())

print(df.describe())

print(df.columns)
```

```
366 LP002989 Male No 0 Graduate Yes
ApplicantIncome CoapplicantIncome LoanAmount Loan_Amount_Term \
362 4009 1777 113.0 360.0
363 4158 709 115.0 360.0
364 3250 1993 126.0 360.0
365 5000 2393 158.0 360.0
366 9200 0 98.0 180.0

Credit_History Property_Area
362 1.0 Urban
363 1.0 Urban
364 NaN Semiurban
365 1.0 Rural
366 1.0 Rural
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 367 entries, 0 to 366
Data columns (total 12 columns):
# Column Non-Null Count Dtype
---
0 Loan_ID 367 non-null object
1 Gender 366 non-null object
2 Married 367 non-null object
3 Dependents 357 non-null object
4 Education 367 non-null object
5 Self_Employed 344 non-null object
6 ApplicantIncome 367 non-null int64
7 CoapplicantIncome 367 non-null int64
8 LoanAmount 362 non-null float64
9 Loan_Amount_Term 361 non-null float64
10 Credit_History 338 non-null float64
11 Property_Area 367 non-null object
dtypes: float64(3), int64(2), object(7)
memory usage: 34.5+ KB
None
```

```

ApplicantIncome  CoapplicantIncome  LoanAmount  Loan_Amount_Term \
count      367.000000      367.000000  362.000000      361.000000
mean      4805.599455      1569.577657  136.132597      342.537396
std      4910.685399      2334.232099    61.366652      65.156643
min         0.000000         0.000000    28.000000        6.000000
25%      2864.000000         0.000000   100.250000      360.000000
50%      3786.000000      1025.000000   125.000000      360.000000
75%      5060.000000      2430.500000   158.000000      360.000000
max      72529.000000     24000.000000   550.000000     480.000000

Credit_History
count      338.000000
mean         0.825444
std         0.380150
min         0.000000
25%         1.000000
50%         1.000000
75%         1.000000
max         1.000000
Index(['Loan_ID', 'Gender', 'Married', 'Dependents', 'Education',
       'Self_Employed', 'ApplicantIncome', 'CoapplicantIncome', 'LoanAmount',
       'Loan_Amount_Term', 'Credit_History', 'Property_Area'],
      dtype='object')

```

```

categorical_cols = ['Gender', 'Married', 'Dependents', 'Education',
'Self_Employed', 'Property_Area']

```

```

for col in categorical_cols:

```

```

    unique_vals = df[col].unique()

```

```

    print(f"\nUnique values in '{col}': {unique_vals}")

```

```

Unique values in 'Gender': ['Male' 'Female' nan]

Unique values in 'Married': ['Yes' 'No']

Unique values in 'Dependents': ['0' '1' '2' '3+' nan]

Unique values in 'Education': ['Graduate' 'Not Graduate']

Unique values in 'Self_Employed': ['No' 'Yes' nan]

Unique values in 'Property_Area': ['Urban' 'Semiurban' 'Rural']

```

```

df['Dependents'] = df['Dependents'].replace('3+', 3)

```

```

df['Dependents'] = pd.to_numeric(df['Dependents'],
errors='coerce').astype('Int64')

```

```

print("\nValue counts in 'Dependents' after conversion:")

```

```
print(df['Dependents'].value_counts(dropna=False))
```

```
Value counts in 'Dependents' after conversion:
Dependents
0      200
2       59
1       58
3       40
<NA>   10
Name: count, dtype: Int64
```

```
cols = ['ApplicantIncome', 'CoapplicantIncome', 'LoanAmount']
```

```
for col in cols:
```

```
    print(f"\nStatistics for '{col}':")
```

```
    print(f"Mean: {df[col].mean():.2f}")
```

```
    print(f"Median: {df[col].median():.2f}")
```

```
    print(f"Mode: {df[col].mode().values}")
```

```
    print(f"Range: {df[col].max() - df[col].min():.2f}")
```

```
    print(f"Variance: {df[col].var():.2f}")
```

```
    print(f"Standard Deviation: {df[col].std():.2f}")
```

```
Statistics for 'ApplicantIncome':
Mean: 4805.60
Median: 3786.00
Mode: [3500 5000]
Range: 72529.00
Variance: 24114831.09
Standard Deviation: 4910.69

Statistics for 'CoapplicantIncome':
Mean: 1569.58
Median: 1025.00
Mode: [0]
Range: 24000.00
Variance: 5448639.49
Standard Deviation: 2334.23

Statistics for 'LoanAmount':
Mean: 136.13
Median: 125.00
Mode: [150.]
Range: 522.00
Variance: 3765.87
Standard Deviation: 61.37
```

RESULT:

Thus, the given program was written and executed successfully.